SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	P H Devesh
Register Number	1911260236
Course Code & Name	CSA0209 & C Programming
Slot	B
Assignment / Task Title	C03 · Searching, Sorting and Data Processing
GitHub Repository URL	https://github.com/DeveshPH/C-CSA0209
Submission Date	_____

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

CSA0209 – C PROGRAMMING

C03 – Searching, Sorting and Data Processing

Assignment solutions based on the supplied SIMATS Online Programming Lab screenshots. Includes algorithm design, completed C programs, test cases and complexity analysis.

Evidence note: The screenshots do not provide actual machine timing measurements or a complete rainfall dataset. Timing comparisons are therefore explained using complexity and expected behaviour; the sample data below is illustrative.

Contents

Q1 Searching a Library Catalogue
Q2 Sorting the Examination Hall Seating Register
Q3 Merging Two Merit Lists
Q4 Two-Dimensional Arrays: District Rainfall Records
Q5 Processing Text Without the Library
Q6 A Directory of Names: Sorting and Searching Strings
Q7 Design an Election Counting Program

Q1 – Searching a Library Catalogue (6 Marks)

Algorithms and analysis

Linear search checks each element in order and works on unsorted data. Its best, average and worst comparison counts are 1, (n+1)/2 and n. **Binary search** requires ascending sorted data and halves the remaining range after each comparison. Its best case is 1 and its worst case is floor(log2 n)+1 comparisons.

```
BINARY-SEARCH(a, n, key):
    low = 0; high = n - 1; steps = 0
    while low <= high:
        mid = low + (high - low) / 2
        steps++
        if a[mid] == key: return mid, steps
        if key < a[mid]: high = mid - 1
        else: low = mid + 1
    return -1, steps
```

Completed C program

```
#include <stdio.h>
int steps;
int binary_search(const int a[], int n, int key) {
    int low=0, high=n-1;
    steps=0;
    while(low<=high) {
        int mid=low+(high-low)/2;
        steps++;
        if(a[mid]==key) return mid;
        if(key<a[mid]) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
int main(void) {
    int a[]={1021,1048,1103,1190,1222,1305,1377,1402,1460,1533};
    int keys[]={1021,1305,1533,1400};
    int n=sizeof a/sizeof a[0];
    for(int k=0;k<4;k++) {
        int p=binary_search(a,n,keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n",keys[k],p,steps);
    }
    return 0;
}
```

```
Expected output:
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```

The missing key needs four comparisons because binary search must eliminate every remaining interval before concluding that the key is absent. For 100,000 items, linear search needs up to 100,000 comparisons (about 50,000 on average for a uniformly positioned successful key); binary search needs at most 17. The midpoint expression low + (high-low)/2 avoids overflow in low+high.

Method	Average time	Worst time	Extra space
Linear search	O(n)	O(n)	O(1)

Method	Average time	Worst time	Extra space
Binary search	O(log n)	O(log n)	O(1) iterative
Hash table, linear probing	O(1) average	O(n)	O(n)
Interpolation search	O(log log n) for uniform keys	O(n)	O(1)

Keeping data sorted is worthwhile when searches are frequent and additions are relatively rare. Inserting into a sorted array can shift O(n) values. Compare the number of daily insertions with the number of searches; a balanced tree or hash table may be preferable for frequent updates. Test n=1,000, 10,000 and 100,000 using successful and missing keys and record comparisons separately from elapsed time.

Q2 – Sorting the Examination Hall Seating Register (7 Marks)

Algorithm comparison

Algorithm	Best	Average / worst	Stable?
Bubble sort + early exit	O(n)	O(n²)	Yes
Selection sort	O(n²)	O(n²)	No, usual swap version
Insertion sort	O(n)	O(n²)	Yes

For the six-value example [42, 17, 35, 12, 29, 8], each method produces [8, 12, 17, 29, 35, 42]. Bubble sort swaps adjacent inversions; selection sort repeatedly selects the smallest remaining item; insertion sort inserts each next item into the sorted prefix. Stability matters when equal marks must retain their earlier registration order.

C program with comparison and movement counters

```
#include <stdio.h>
#define N 6
long long cmp, moves;
void bubble(int a[],int n){
 cmp=moves=0;
 for(int i=0;i<n-1;i++){
  int changed=0;
  for(int j=0;j<n-1-i;j++){
   cmp++;
   if(a[j]>a[j+1]){
    int t=a[j];a[j]=a[j+1];a[j+1]=t;moves++;changed=1;
   }
  }
  if(!changed) break;
 }
}
void selection(int a[],int n){
 cmp=moves=0;
 for(int i=0;i<n-1;i++){
  int m=i;
  for(int j=i+1;j<n;j++){cmp++;if(a[j]<a[m])m=j;}
  if(m!=i){int t=a[i];a[i]=a[m];a[m]=t;moves++;}
 }
}
void insertion(int a[],int n){
 cmp=moves=0;
 for(int i=1;i<n;i++){
  int key=a[i],j=i-1;
  while(j>=0){cmp++;if(a[j]<=key)break;a[j+1]=a[j];moves++;j--;}
  a[j+1]=key;
 }
}
void show(int a[],int n){
 for(int i=0;i<n;i++)printf("%d ",a[i]);
 printf("\ncomparisons=%lld movements=%lld\n",cmp,moves);
}
int main(void){
 int original[N]={42,17,35,12,29,8},a[N];
 for(int i=0;i<N;i++)a[i]=original[i];bubble(a,N);puts("Bubble:");show(a,N);
 for(int i=0;i<N;i++)a[i]=original[i];selection(a,N);puts("Selection:");show(a,N);
 for(int i=0;i<N;i++)a[i]=original[i];insertion(a,N);puts("Insertion:");show(a,N);
 return 0;
}
```

Early-exit bubble sort stops after a pass with no swaps. Insertion sort is fast on nearly sorted data because only a few shifts are required. Reverse-sorted input causes quadratic work. Selection sort always makes n(n-1)/2 comparisons, regardless of input order. For a register that is sorted except for 20 late registrations, insertion sort is a suitable simple choice. Counting sort is O(n+k) time and O(k) space when integer keys lie in a compact range of size k; it is unsuitable when k is huge or keys are not bounded integers.

Q3 – Merging Two Merit Lists (7 Marks)

Merge algorithm

The two input lists are sorted in descending order. Compare the current values, append the larger one, and advance its index. Once one list ends, copy the remaining values from the other list.

```
#include <stdio.h>
int merge(const int a[],int na,const int b[],int nb,int out[]){
 int i=0,j=0,k=0;
 while(i<na && j<nb){
  if(a[i]>=b[j]) out[k++]=a[i++];
  else out[k++]=b[j++];
 }
 while(i<na) out[k++]=a[i++];
 while(j<nb) out[k++]=b[j++];
 return k;
}
int main(void){
 int a[]={98,91,85,72},b[]={95,91,80},out[7];
 int n=merge(a,4,b,3,out);
 for(int i=0;i<n;i++)printf("%d ",out[i]);
 printf("\n"); return 0;
}
```

Expected output: 98 95 91 91 85 80 72. At most m+n-1 comparisons are made in the main loop because each comparison advances at least one input index. Using >= selects campus A first on equal scores, preserving that campus's equal-score priority. If one list is empty, the main loop is skipped and the other list is copied.

Approach	Time	Extra space
Merge sorted lists	O(m+n)	O(m+n) output
Concatenate then qsort	O((m+n) log(m+n))	Implementation-dependent
Simple in-place shifting merge	Can be O(m·n)	O(1)

The third array is usually preferred for its speed and simple correctness. In-place merging is useful when memory is severely limited, but is more complicated and can require many shifts.

Q4 – Two-Dimensional Arrays: District Rainfall Records (7 Marks)

Memory layout and calculations

C stores rain[38][12] in row-major order: the 12 months of district 0 are contiguous, followed by district 1, etc. For base address B and element width w, address(rain[i][j]) = B + ((i×12)+j)×w. Row-wise traversal follows adjacent memory and is normally more cache-friendly than column-wise traversal. Both access forms are O(1), but locality changes practical speed.

```
#include <stdio.h>
#define D 38
#define M 12
void annual_totals(const double r[D][M],double total[D]){
 for(int i=0;i<D;i++){total[i]=0;for(int j=0;j<M;j++)total[i]+=r[i][j];}
}
void monthly_averages(const double r[D][M],double avg[M]){
 for(int j=0;j<M;j++){double s=0;for(int i=0;i<D;i++)s+=r[i][j];avg[j]=s/D;}
}
void wettest(const double r[D][M],int *di,int *mo,double *mx){
 *mx=r[0][0];*di=0;*mo=0;
 for(int i=0;i<D;i++)for(int j=0;j<M;j++)
  if(r[i][j]>*mx){*mx=r[i][j];*di=i;*mo=j;}
}
void deficient(const double r[D][M],double state_monthly_avg){
 double limit=.60*12*state_monthly_avg;
 for(int i=0;i<D;i++){
  double total=0;for(int j=0;j<M;j++)total+=r[i][j];
  if(total<limit)printf("District %d: %.2f mm\n",i+1,total);
 }
}
int main(void){
 static double rain[D][M]={{0}};
 double total[D],avg[M],mx;int di,mo;
 annual_totals(rain,total);monthly_averages(rain,avg);
 wettest(rain,&di,&mo,&mx);
 printf("Wettest district %d month %d: %.2f mm\n",di+1,mo+1,mx);
 return 0;
}
```

The state monthly average is total rainfall divided by 38×12. A district is deficient when its annual total is below 60% of 12 times that state monthly average. For a hand-checkable 3×4 array {{10,20,30,40},{5,15,25,35},{8,18,28,38}}, row totals are 100, 80 and 92; column averages are 23/3, 53/3, 83/3 and 113/3; the maximum is 40 at district-row 1, month 4.

A 3-D array rain[years][districts][months] adds a year dimension. A flat 1-D array uses index i×12+j and retains O(1) access without row-pointer overhead. An array of separately allocated row pointers adds O(38) pointer space and can fragment memory. All representations store the same 456 values, but contiguous storage usually has better locality.

Q5 – Processing Text Without the Library (6 Marks)

String concepts and completed functions

A C string ends with the null character '\\0'. A new word begins when a non-space character follows the start of the string or a whitespace character. char s[] = "OK" creates a modifiable array; char *s = "OK" points to a string literal and must not be modified.

```
#include <stdio.h>
#include <ctype.h>
int word_count(const char s[]){
 int count=0,inside=0;
 for(int i=0;s[i]!='\0';i++){
  if(isspace((unsigned char)s[i])) inside=0;
  else if(!inside){count++;inside=1;}
 }
 return count;
}
int is_palindrome(const char s[]){
 int l=0,r=0;
 while(s[r]!='\0')r++; r--;
 while(l<r){
  while(l<r && !isalpha((unsigned char)s[l]))l++;
  while(l<r && !isalpha((unsigned char)s[r]))r--;
  if(tolower((unsigned char)s[l])!=tolower((unsigned char)s[r]))return 0;
  l++;r--;
 }
 return 1;
}
void uppercase(char s[]){
 for(int i=0;s[i]!='\0';i++)s[i]=(char)toupper((unsigned char)s[i]);
}
void reverse(char s[]){
 int i=0,j=0;while(s[j]!='\0')j--; /* corrected below */
}
```

Use this corrected reverse function (the loop must advance to the null terminator, then step back one position):

```
void reverse(char s[]){
 int i=0,j=0;
 while(s[j]!='\0')j++;
 j--;
 while(i<j){char t=s[i];s[i]=s[j];s[j]=t;i++;j--;}
}
int main(void){
 char msg[]="Meet at the library at 5 PM";
 printf("Words: %d\n",word_count(" Meet at the library at 5 PM "));
 printf("%d %d\n",is_palindrome("Madam, I'm Adam"),is_palindrome("Hello"));
 uppercase(msg);printf("%s\n",msg);reverse(msg);printf("%s\n",msg);
 return 0;
}
```

Expected key output: Words: 7 and palindrome results 1 0. isspace() counts tabs as separators; if only ordinary spaces are intended, check specifically for ' '. strlen() in every loop condition can cause O(n^2) work; calculate the length once or scan to '\\0'. Iterative reversal is O(n) time and O(1) extra space; recursive reversal uses O(n) stack space.

string.h supplies strlen, strcpy and strcmp; ctype.h supplies toupper, tolower, isspace and isalpha. strcpy can overflow a destination that is too small. Prefer fgets for input and snprintf or carefully bounded copies. gets() was removed because it cannot limit input length and can overflow the buffer.

Q6 – A Directory of Names: Sorting and Searching Strings (7 Marks)

Design and program

strcmp compares bytes until a mismatch or the null terminator; it returns negative, zero or positive. Uppercase and lowercase letters have different character codes, so ordinary strcmp is case-sensitive. A case-insensitive comparison converts characters using tolower(). Sorting makes duplicate names adjacent.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 500
#define LEN 31
int cmpname(const char *a,const char *b){
 while(*a && *b){
  int x=tolower((unsigned char)*a),y=tolower((unsigned char)*b);
  if(x!=y)return x-y;a++;b++;
 }
 return tolower((unsigned char)*a)-tolower((unsigned char)*b);
}
```

```
void sort_names(char a[][LEN],int n){
 for(int i=1;i<n;i++){
  char key[LEN];strcpy(key,a[i]);int j=i-1;
  while(j>=0 && cmpname(a[j],key)>0){strcpy(a[j+1],a[j]);j--;}
  strcpy(a[j+1],key);
 }
}
int binary_find(char a[][LEN],int n,const char *key){
 int lo=0,hi=n-1;
 while(lo<=hi){
  int mid=lo+(hi-lo)/2,c=cmpname(a[mid],key);
  if(c==0)return mid;if(c<0)lo=mid+1;else hi=mid-1;
 }
 return -1;
}
int main(void){
 char names[MAX][LEN],key[LEN];int n;
 if(scanf("%d",&n)!=1 || n<1 || n>MAX)return 1;
 getchar();
 for(int i=0;i<n;i++){
  if(!fgets(names[i],LEN,stdin))return 1;
  names[i][strcspn(names[i],"\n")]='\0';
 }
 sort_names(names,n);
 for(int i=0;i<n;i++)puts(names[i]);
 puts("Duplicates:");
 for(int i=1;i<n;i++)if(cmpname(names[i-1],names[i])==0)puts(names[i]);
 if(fgets(key,LEN,stdin)){
  key[strcspn(key,"\n")]='\0';
  printf("Search index: %d\n",binary_find(names,n,key));
 }
 return 0;
}
```

The array uses LEN=31 so it can store 30 visible characters plus '\\0'. With LEN=30, only 29 visible characters fit. fgets reads at most size−1 characters. Insertion sorting names costs $O(n^2L)$ in the worst case because comparisons and copies may inspect up to L characters. Binary search after sorting costs O(L log n). Sorting indices or pointers reduces the cost of swaps, but adds O(n) pointer/index storage and indirect access. qsort typically uses O(n log n) comparisons, each potentially O(L).

Q7 – Design an Election Counting Program from First Principles (10 Marks)

Requirements and pseudocode

Inputs are candidate count c (1–12), vote count v (up to the test limit), and v candidate numbers. A vote is valid when $1 \le vote \le c$. Maintain count[c+1], valid and invalid counters, and an order[c] array. Sort candidate identifiers by their count in descending order, then detect all candidates tied at the maximum.

```
#include <stdio.h>
#define MAXC 12
#define MAXV 500000
int main(void){
 int c,v,count[MAXC+1]={0},valid=0,invalid=0,order[MAXC];
 if(scanf("%d%d",&c,&v)!=2 || c<1 || c>MAXC || v<0 || v>MAXV)return 1;
 for(int i=0;i<v;i++){
  int vote;if(scanf("%d",&vote)!=1)return 1;
  if(vote>=1 && vote<=c){count[vote]++;valid++;}
  else invalid++;
 }
 for(int i=0;i<c;i++)order[i]=i+1;
 for(int i=1;i<c;i++){
  int key=order[i],j=i-1;
  while(j>=0 && count[order[j]]<count[key]){order[j+1]=order[j];j--;}
  order[j+1]=key;
 }
 printf("Valid votes: %d\nInvalid votes: %d\n",valid,invalid);
 puts("Candidate totals (descending):");
 for(int i=0;i<c;i++)printf("Candidate %d: %d\n",order[i],count[order[i]]);
 int high=count[order[0]],winners=0;
 for(int i=1;i<=c;i++)if(count[i]==high)winners++;
 if(winners>1){
  printf("Tie:");for(int i=1;i<=c;i++)if(count[i]==high)printf(" %d",i);
  printf(" (%d votes each)\n",high);
 }else printf("Winner: Candidate %d\n",order[0]);
 return 0;
}
```

Hand-check: c=3 and votes [1,2,2,3,0,4,-1,1] gives 6 valid and 2 invalid votes. Candidate totals are 1:2, 2:2, 3:1; candidates 1 and 2 tie.

Method	Time	Space
Count array + candidate ordering	$O(v+c^2)$ with insertion ordering	O(c)
Search candidate list for every vote	$O(v \cdot c)$	O(c)
Sort all votes then count equal runs	O(v log v)	O(v)

A count array is efficient because each valid vote updates one indexed counter in O(1). For 500 candidates, enlarge the arrays and use an O(c log c) sort if necessary. Limitations include lack of voter authentication, duplicate-vote prevention and audit security; production election systems require additional safeguards.

Overall conclusion

The assignment shows that data order, memory layout and representation determine efficiency. Binary search exploits sorted arrays; insertion sort suits nearly sorted registers; merging uses existing order; row-major arrays improve cache locality; safe string processing avoids buffer errors; and count arrays provide efficient election totals for small integer candidate identifiers.